Contrato de Servicio: Shared File Service

1. Información general

Campo	Valor
Nombre del servicio	Shared File Service
Descripción	Servicio de negocio orientado al usuario final para compartir archivos y directorios, gestionar permisos estilo Unix sobre ellos, y navegar el propio Home. No almacena bytes: delega la persistencia física en Distributed Storage Service y añade la semántica de compartición.
Objetivo	Permitir que un usuario comparta contenido de su Home con otros usuarios o grupos, con control fino de permisos, sin que cada consumidor tenga que reimplementar esa lógica sobre el Storage.
Responsabilidad	Compartición de archivos/directorios, gestión de permisos Unix-style sobre elementos compartidos, listado de "compartido conmigo" / "que yo comparto", gestión del Home a nivel de negocio (no física).
Tecnología	Libre para el equipo; el contrato es independiente de la implementación.
Protocolo de comunicación	HTTP/REST
Servicios consumidores	Clientes (Web, Windows, Linux, Android, iOS).
Servicios proveedores	Authentication Service (`ValidarToken`, `ObtenerPermisos`), Distributed Storage Service (`SubirArchivo`, `DescargarArchivo`, `ListarArchivos`, `ObtenerMetadatos`, `ObtenerCuotaUsuario` — se delega, no se duplica).

2. Funcionalidades (operaciones expuestas)

1. SubirArchivo (fachada sobre Storage, añade validación de contexto de Home)
2. DescargarArchivo (fachada sobre Storage, valida permisos de compartición)
3. CompartirArchivo
4. CompartirDirectorio
5. DejarDeCompartir
6. ActualizarPermisosCompartidos

7. ListarCompartidosConmigo
8. ListarQueYoComparto
9. ObtenerPermisosEfectivos
10. AceptarInvitacionCompartir
11. RechazarInvitacionCompartir
12. ObtenerHome
13. ListarContenidoHome
14. ObtenerCuotaUsuario (fachada sobre Storage)

---

3. Contrato de cada operación

3.1 CompartirArchivo

Nombre: `CompartirArchivo` **Descripción:** Otorga permisos de acceso sobre un archivo específico a otro usuario o a un grupo. **Protocolo:** HTTP/REST

Parámetros de entrada:

Nombre	Tipo	Obligatorio	Descripción	Restricciones/Validaciones
`token`	string	Sí	Token del propietario o de quien tiene permiso de compartir	Validado contra Authentication Service
`archivoId`	UUID	Sí	Archivo a compartir	Debe existir en Storage y el solicitante debe tener permiso `compartir` sobre él
`destinatarioTipo`	enum	Sí	`USUARIO` o `GRUPO`	—
`destinatarioId`	UUID	Sí	Id del usuario o grupo destinatario	Debe existir en Authentication Service
`nivelPermiso`	enum	Sí	`LECTURA`, `LECTURA_ESCRITURA`, `LECTURA_ESCRITURA_COMPARTIR`	Estilo Unix simplificado: el tercer nivel permite que el destinatario a su vez comparta con otros
`expiracion`	datetime	No	Fecha en que el permiso se revoca automáticamente	Si se omite, el permiso es indefinido hasta revocación manual

Datos de salida:

Nombre	Tipo	Descripción
`compartidoId`	UUID	Identificador de la relación de compartición creada
`fechaCompartido`	datetime	—
`estadoInvitacion`	enum	`PENDIENTE` (si se requiere aceptación) o `ACTIVO` (si es automático)

Posibles errores:

Código	Descripción	Posible solución
`SHR-001`	Archivo no encontrado o no pertenece al solicitante	Verificar `archivoId` y propiedad
`SHR-002`	Solicitante no tiene permiso `compartir` sobre el archivo	Solicitar al propietario original
`SHR-003`	Destinatario (usuario o grupo) no existe	Verificar `destinatarioId`
`SHR-004`	El archivo ya está compartido con ese destinatario	Usar `ActualizarPermisosCompartidos` en lugar de crear uno nuevo
`STG-103` (propagado)	Storage rechaza el acceso por permisos inconsistentes	Sincronizar estado con Storage

Reglas de negocio:

- Compartir un archivo **no mueve ni copia** el contenido; crea una relación de permiso adicional que Storage debe respetar en sus propias validaciones de lectura/escritura (Shared File Service es quien decide "quién puede", pero es Storage quien sigue siendo la fuente de verdad de los bytes).
- Si `nivelPermiso` es `LECTURA_ESCRITURA_COMPARTIR`, el destinatario puede volver a compartir el archivo — esto se valida de forma recursiva contra la cadena de permisos, no contra el propietario original únicamente.
- Compartir con un **grupo** aplica el permiso a todos los miembros actuales; nuevos miembros del grupo heredan el permiso automáticamente (delegado en el modelo de grupos de Authentication).

Seguridad: Requiere token válido. Requiere permiso `compartir` sobre el archivo (que puede tenerlo el propietario original o alguien a quien se le otorgó explícitamente ese nivel).

Dependencias: Authentication Service (validar token, validar existencia de destinatario), Distributed Storage Service (verificar que el archivo existe y aplicar el permiso en sus validaciones de acceso).

Flujo interno:

1. Presentación recibe la solicitud.
2. Aplicación valida token contra Authentication Service.
3. Dominio verifica que el solicitante tiene permiso `compartir` sobre el archivo (consulta a Storage o a la tabla propia de permisos, según diseño).
4. Dominio valida que el destinatario existe (consulta a Authentication).
5. Persistencia registra la relación de compartición.
6. Se notifica a Distributed Storage Service el nuevo permiso para que lo aplique en sus propias validaciones de acceso (o Storage consulta a este servicio en tiempo real — decisión de diseño a definir entre ambos equipos, documentada como dependencia bidireccional).
7. Se emite evento `ArchivoCompartido`.
8. Respuesta al cliente.

3.2 CompartirDirectorio

Mismo patrón que `CompartirArchivo`, con una diferencia de regla de negocio central:

Regla de negocio: el permiso se aplica recursivamente a todo el contenido actual del directorio, **y también al contenido futuro** (todo archivo nuevo agregado al directorio hereda el permiso de compartición del directorio). Esto es distinto de compartir archivo por archivo y debe documentarse explícitamente para que Storage lo respete al crear nuevos archivos dentro de un directorio compartido.

Error adicional: `SHR-101` (directorio no encontrado o vacío de forma inesperada — condición de carrera con una eliminación concurrente).

3.3 DejarDeCompartir

Entrada: `token`, `compartidoId` (o `archivoId` + `destinatarioId`). **Salida:** confirmación. **Errores:** `SHR-001` (relación no encontrada), `SHR-002` (solo el propietario original o quien tenga `compartir` puede revocar). **Regla de negocio:** revocar la compartición de un directorio revoca también el acceso heredado a su contenido, de forma inmediata (no esperar a la siguiente sincronización).

3.4 ActualizarPermisosCompartidos

Entrada: `token`, `compartidoId`, `nuevoNivelPermiso`. **Salida:** confirmación con el nuevo nivel. **Error:** `SHR-002` (permiso insuficiente para modificar), `SHR-201` (intento de degradar

un permiso que ya fue usado para volver a compartir con terceros — regla de negocio: si un destinatario ya compartió con otros usando el nivel `COMPARTIR`, degradarlo no revoca automáticamente esas subcomparticiones; se debe advertir explícitamente al solicitante).

3.5 ListarCompartidosConmigo / ListarQueYoComparto

Entrada: `token`, `paginacion`. **Salida:** lista de `RelacionCompartido` con metadatos del archivo/directorio, nivel de permiso, propietario/destinatario y fecha. **Regla de negocio:** `ListarCompartidosConmigo` excluye automáticamente relaciones expiradas o revocadas; no es responsabilidad del cliente filtrarlas.

3.6 ObtenerPermisosEfectivos

Descripción: Dado un archivo, calcula el permiso resultante de un usuario considerando propiedad directa + compartición individual + compartición por grupo + herencia de directorio. **Entrada:** `token`, `archivoId`, `usuarioObjetivo` (opcional, por defecto el propio solicitante). **Salida:** `nivelPermisoEfectivo`. **Regla de negocio:** cuando hay múltiples fuentes de permiso (ej. compartido individualmente con `LECTURA` y por grupo con `LECTURA_ESCRITURA`), **prevalece el permiso más permisivo** — esta regla debe ser idéntica a la que aplica Storage internamente, para evitar inconsistencias entre "lo que este servicio dice que puedo hacer" y "lo que Storage realmente permite".

3.7 AceptarInvitacionCompartir / RechazarInvitacionCompartir

Aplica solo si la política institucional exige aceptación explícita (en lugar de compartición automática). **Entrada:** `token`, `compartidoId`. **Salida:** confirmación con `estadoInvitacion` actualizado (`ACTIVO` o `RECHAZADO`). **Regla de negocio:** mientras la invitación esté `PENDIENTE`, el destinatario no tiene acceso real, aunque la relación ya exista en el modelo de datos.

3.8 ObtenerHome / ListarContenidoHome

Entrada: `token`. **Salida:** `homePath` (delegado de Authentication) + listado del contenido de primer nivel (delegado de Storage vía `ListarArchivos`). **Regla de negocio:** este servicio nunca almacena la ruta del Home de forma independiente — siempre la obtiene de Authentication (fuente de verdad) para evitar desincronización si el Home se reubica.

---

4. Modelo de datos

RelacionCompartido

Atributo	Tipo	Descripción	Restricciones
`id`	UUID	Identificador	PK
`elementoId`	UUID	Archivo o directorio compartido	FK lógica a Storage
`elementoTipo`	enum	`ARCHIVO`, `DIRECTORIO`	—
`propietarioId`	UUID	Quien comparte	FK lógica a Authentication
`destinatarioTipo`	enum	`USUARIO`, `GRUPO`	—
`destinatarioId`	UUID	A quién se comparte	FK lógica a Authentication
`nivelPermiso`	enum	`LECTURA`, `LECTURA_ESCRITURA`, `LECTURA_ESCRITURA_COMPARTIR`	—
`estadoInvitacion`	enum	`PENDIENTE`, `ACTIVO`, `RECHAZADO`, `REVOCADO`	—
`fechaCreacion`	datetime	—	Inmutable
`fechaExpiracion`	datetime (nullable)	—	—

PermisoEfectivo (objeto calculado, no persistido)

Atributo	Tipo	Descripción
`usuarioId`	UUID	—
`elementoId`	UUID	—
`nivelResultante`	enum	Resultado del cálculo de fuentes múltiples
`fuentes`	array<RelacionCompartido>	Trazabilidad de por qué se obtuvo ese nivel

5. Eventos que genera el servicio

Evento	Disparado cuando	Consumidores típicos
`ArchivoCompartido`	`CompartirArchivo`/`CompartirDirectorio` exitoso	Monitoring Service (métricas de uso), cliente destinatario (notificación)
`ComparticionRevocada`	`DejarDeCompartir`	Monitoring Service
`InvitacionPendiente`	Compartición que requiere aceptación	Cliente destinatario (notificación)

---

6. Integración

- **Quién consume:** todos los clientes finales (Web, Windows, Linux, Android, iOS) — nunca otro servicio de backend consume este contrato directamente, ya que la semántica de compartición es de cara al usuario.
- **Quién responde:** Shared File Service, delegando en Storage para todo lo relativo a bytes/metadatos físicos y en Authentication para identidad/existencia de destinatarios.
- **Protocolo:** HTTP/REST.
- **Momento del flujo:** siempre después de Authentication (para identidad) y coordinado con Storage (para que los permisos de compartición se reflejen en las validaciones de acceso reales a los archivos).

---

7. Consideraciones de escalabilidad

- **Qué puede crecer:** `ObtenerPermisosEfectivos` es potencialmente costosa si un usuario pertenece a muchos grupos y tiene muchas comparticiones individuales — candidata a cachear con invalidación por evento (`RolAsignado`, `ComparticionRevocada`, etc.).
- **Cuellos de botella posibles:** si Storage consulta a Shared File Service en tiempo real por cada lectura para verificar permisos de compartición, ese round-trip se vuelve costoso a escala — se recomienda que Storage cachee o reciba de forma asíncrona los cambios de permisos relevantes en lugar de consultarlos sincrónicamente en cada descarga.
- **Mejoras futuras:** enlaces de compartición públicos con expiración (fuera del alcance mínimo); notificaciones push en tiempo real de nuevas comparticiones.

---

8. Casos límite

Operación	Caso límite	Comportamiento esperado
`CompartirArchivo`	Se comparte un archivo que es eliminado inmediatamente después por el propietario	La relación de compartición queda huérfana; `ListarCompartidosConmigo` debe filtrarla al detectar que el archivo ya no existe en Storage
`CompartirDirectorio`	Se agrega un archivo nuevo al directorio justo cuando se está revocando la compartición	Debe primar la revocación — el archivo nuevo no debe quedar con acceso heredado de una relación ya revocada
`ActualizarPermisosCompartidos`	Se degrada un permiso mientras el destinatario está descargando el archivo en ese instante	La descarga en curso se completa (no se interrumpe a mitad de transferencia), el nuevo nivel aplica desde la siguiente operación
`ObtenerPermisosEfectivos`	Un usuario pertenece a un grupo que fue eliminado a mitad del cálculo	El cálculo debe ignorar de forma segura las fuentes inválidas, nunca fallar por completo
`CompartirArchivo` con grupo	El grupo no tiene miembros	Se permite (la compartición queda "vacía" hasta que se agreguen miembros), no es un error

Contrato listo para implementación independiente de Shared File Service. Siguiente en la secuencia: File Sync Service.